

Advanced React Development — Lab Manual

Version 2.1 — May 2026

Chris Minnick

May 2026

Table of Contents

Advanced React Development — Lab Manual

Version: 2.1.0

Author: Chris Minnick · WatzThis

Date: May 2026

About this manual

This manual collects the eight hands-on labs from the Advanced React Development course, version 2.1. Each lab is self-contained and follows the same structure: goal, time budget, setup, numbered phases, optional stretch task, and hints for when you get stuck.

The labs live in `lab-files/lab-01` through `lab-files/lab-08` in the course repository, which is already cloned to your VM at the start of class. Each lab folder is self-contained — it has its own copy of every project the lab needs, with whatever setup that lab assumes.

For each lab, you'll typically:

1. Open `lab-files/lab-NN/` in your editor.
2. Make a backup copy of the folder before you start changing things, so you can recover if you wreck the codebase past saving.
3. Work through the lab phases in order, editing files in place.
4. Compare your finished work against the reference solution in `solutions/lab-NN-*/`.

If you finish early, every lab has a stretch task. They're worth doing — the stretch sections preview Day 2 / Day 3 material from a different angle.

How the eight labs fit together

Lab	Day	Time	Focus
Lab 1 — Modernize the Chat App	1	90 min	Class components → React 19 + hooks; AI review
Lab 2 — Routing + Auth in Two Frameworks	1	105 min	RR v7 framework mode + Next.js App Router
Lab 3 — Pick-Your-Tool State Refactor	2	90 min	Zustand / Jotai / RTK + 60-second pitch
Lab 4 — TanStack Query Migration	2	75 min	Replace fetch+useEffect; optimistic updates
Lab 5 — Server Components Dashboard	2	75 min	Server Components + Server Actions; boundary review
Lab 6 — Profile and Optimize	3	75 min	Measurement first; three optimizations w/ numbers
Lab 7 — Test Suite from Scratch	3	75 min	Vitest + RTL + MSW; review AI-generated tests
Lab 8 — AI-Assisted Feature, End to End	3	90 min	Capstone: build w/ AI, self-review

Total lab time across the course: **~11 hours of hands-on work** (more than half of the course). Most labs include a stretch task that adds 15-30 minutes.

Lab 1 — Modernize the Chat App

Time: 90 minutes

Format: Hands-on. Work alone or in pairs.

Goal: Take a deliberately legacy React-18-on-CRA chat app and modernize it to React 19 + Vite + function components + hooks. Use an AI assistant for the first pass, then catch and fix what it got wrong.

What you'll have at the end

A working real-time chat app that:

1. Runs on Vite, not Create React App
2. Uses React 19, not React 18
3. Has zero class components — all function components with hooks
4. Has no `useEffect` for things that aren't synchronization with external systems
5. Comes with a short written review of three mistakes the AI made and how you fixed them

Plus, if you have time: messages that appear instantly thanks to `useOptimistic`.

Time budget

Phase	Minutes
Setup + run the starter	10
Part 1 — Modernize with AI	50
Part 2 — Review the AI's work	30
Stretch (if time): <code>useOptimistic</code>	+

If you finish early, do the stretch task or help a neighbor.

Setup

The course repo is already on your VM at `C:\advanced-react-19` (or `~/advanced-react-19` if you're on Mac/Linux).

Open `lab-files/lab-01/` in your editor. You'll see two top-level folders:

```
lab-files/lab-01/  
├── real-time-chat/      ← chat app (client + chat server)  
└── social-media/server/ ← auth backend
```

Read `lab-files/lab-01/README.md` — it has the three-terminal startup sequence. The short version:

```
# Terminal 1 — auth backend (port 4000)  
cd lab-files/lab-01/social-media/server  
npm install  
npm run dev
```

```
# Terminal 2 — socket.io chat server (port 8081)  
cd lab-files/lab-01/real-time-chat/server  
npm install  
npm start
```

```
# Terminal 3 — chat client (port 3000)  
cd lab-files/lab-01/real-time-chat/client  
npm install  
npm start
```

Open <http://localhost:3000>. Sign up with a fake email, then log in. Open a second browser (Chrome and Firefox both, or Chrome + Edge) and chat with yourself.

If the chat works, you're set. **Make a backup copy of your starter state before you start changing things** — that way if you wreck the codebase past recovery you can copy it back fresh:

From the repo root, in any terminal:

```
cp -R lab-files/lab-01 lab-files/lab-01-backup
```

Windows PowerShell:

```
Copy-Item -Recurse lab-files/lab-01 lab-files/lab-01-backup
```

You'll work directly inside `lab-files/lab-01/real-time-chat/client/` for most of this lab. The chat server doesn't change.

Part 1 — Modernize with AI (45 min)

The starter is intentionally a museum piece: class components, `componentDidMount`, `setState({})`, and `forwardRef`. Your job is to drag it into 2026.

Step 1 — Pick your AI assistant

Use whatever you have. Cursor, Claude Code, Copilot Chat, Windsurf, Codeium — they all work. The lab is tool-agnostic.

Make sure your assistant can read `AGENTS.md` (or `CLAUDE.md`) at the repo root before you start. Those files tell the AI what conventions this codebase follows; without them, the AI defaults to whatever it sees most often online (often: class components and `getByTestId`).

Step 2 — Give the AI the goal

Tell it (or paste a prompt like) something like:

Modernize this React app from React 18 / CRA / class components to React 19 / Vite / function components and hooks. Convert every class component to a function component. Replace `componentDidMount` and `componentWillUnmount` with `useEffect`. Replace `this.setState` with `useState` or `useReducer`. Replace `forwardRef` with the React 19 `ref-as-a-prop` pattern. Don't change the app's behavior; only the implementation. Follow the conventions in `AGENTS.md`.

Don't accept its first attempt blindly. Read what it produced.

Step 3 — Run the converted app

Run `npm run dev` after each AI batch (the modernized client uses Vite on port 5173). Fix runtime errors. Common failure points:

- The AI converted `componentDidMount` to a `useEffect(() => {...}, [])` — but moved a value into the effect that should be in the dep array. You'll see it firing once when it should fire every time, or vice versa.

- The AI wrapped state setters in their own `useEffect`. Often wrong — these should be inside the event handlers that triggered them.
- The AI converted `this.setState({ count: this.state.count + 1 })` to `setCount(count + 1)`. That's race-prone. Use `setCount(n => n + 1)`.
- The AI broke socket event handler cleanup, leading to multiple subscriptions stacking up across re-renders.

You don't need to ship a version that's identical to the AI's first draft — fix what's wrong as you go.

Step 4 — Check off the modernization checklist

Before you move on, your client code should have:

- ☐ No class declarations in `src/`
 - ☐ No `componentDidMount`, `componentWillUnmount`, `componentDidUpdate`, `getDerivedStateFromProps`, `componentDidCatch` anywhere
 - ☐ No `forwardRef` (use `ref` as a regular prop)
 - ☐ No `setState` calls (use `useState` or `useReducer`)
 - ☐ No `useEffect` for derived state — anything you can compute from props/state should be a plain calculation
 - ☐ React 19 in `package.json` (not ^18)
 - ☐ `npm run dev` works; you can sign up, log in, and chat across two browsers
-

Part 2 — Review the AI's work (25 min)

This is the part of the lab the rest of the course depends on. Don't skip it.

Step 1 — Use the checklist from Module 2

Open the slide deck for Module 2 to the “AI hook mistakes — your review checklist” slide. You're looking for:

- `useState` for derived data that should be a plain calculation
- `useEffect` to reset state on prop change (use a key prop instead)
- `useEffect` to fetch data (in this app, it's borderline-OK — but watch for it being misused)
- Stale closure in `useEffect` (missing dep, or use functional `setState`)
- Hooks inside conditionals, callbacks, or after early returns
- Custom hooks called conditionally

Step 2 — Find at least three issues

Find at least three real issues in the AI-generated code. Real issues — not style preferences, not naming. Things that are wrong or fragile.

For each one, write:

1. **Where it is** — file path and line number(s)
2. **What the AI did** — the wrong code, copy-pasted
3. **Why it's wrong** — what's the actual bug or smell
4. **What you replaced it with** — the corrected code

Save these as `lab01-ai-review.md` at the root of `lab-files/lab-01/`.

Stretch task — `useOptimistic` for instant messages

If you finish early, add optimistic message sending so a message appears in the chat thread instantly when the user hits send, instead of waiting for the server round-trip.

You'll need:

```
const [optimisticMessages, addOptimistic] = useOptimistic(
  messages,
  (current, newMessage) => [...current, { ...newMessage, sending: true }]
);
```

Then in your form's submit handler:

```
async function handleSend(formData) {
  const text = formData.get('text');
  addOptimistic({ text, author: currentUser, id: crypto.randomUUID() });
  await sendMessage(text); // socket emit or HTTP post
}
```

Wire `handleSend` as a form action — `<form action={handleSend}>` — so React treats it as a transition. (Without that, the optimistic value reverts on the next render.) Render `optimisticMessages` instead of `messages`. Add a CSS class for `.sending` that dims the bubble until the server confirms.

If the AI helps you with this — review the `useOptimistic` setup carefully. AI tools as of mid-2026 still get the reducer signature wrong about a third of the time.

Future feature ideas (post-lab, if you want to keep playing)

The original “Future Features” section from the 2024 lab is still relevant:

- Direct messages (1:1 instead of single global channel)
- Multiple channels with channel switching
- Typing indicators
- Read receipts
- Image/file uploads

- Emoji reactions
- Message editing and deletion (with audit trail)

These are not part of Lab 1. Pick one over lunch if you want a personal challenge.

Hints if you get stuck

- **I broke something past recovery:** delete `lab-files/lab-01/` and restore from your backup copy.
 - **Vite import errors after the AI's first pass:** the AI may have used `process.env.REACT_APP_X` (CRA convention). Vite uses `import.meta.env.VITE_X`. Update your env names and the references.
 - **Socket.io reconnects on every render:** your `useEffect` is missing the empty dependency array, or you're recreating the socket inside the component body. Move the socket to a `useRef` initialized once, or to a module-scope singleton.
 - **Class-based context provider didn't convert cleanly:** rewrite the provider as a function component using `useState` or `useReducer`, then export it.
 - **Auth `setState({ user })` was conditional inside `componentDidMount`:** the AI probably converted it to a `useState` initialized with a function. Check that the function returns the right initial value.
 - **The chat UI shows the same message twice:** you have two socket subscriptions. The cleanup in `useEffect` isn't firing. Make sure you return `() => socket.off('message', handler)` in the effect.
-

Reference solution

After the lab, the reference solution lives in `solutions/lab-01-modernize/`. Read its `README.md` first; it explains what changed and why. Don't peek before you've done your own review — the value of the lab is in finding the bugs yourself.

Lab 2 — Routing + Auth in Two Frameworks

Time: 105 minutes

Format: Hands-on. Work alone or in pairs.

Goal: Implement the same authenticated routing — `/login`, `/signup`, `/logout`, and a protected `/home` — twice. First in React Router v7 framework mode, then port the same feature to Next.js 15 App Router. Reflect on what differs and what doesn't.

What you'll have at the end

Two working clients that share one backend:

1. A **React Router v7** client at `lab-files/lab-02/social-media-rr-v7/` with login, signup, logout, and a protected home page that lists posts.
2. A **Next.js 15** client at `lab-files/lab-02/social-media-nextjs/` with the same feature set, using middleware-based auth.
3. A short written reflection comparing the two — saved as `lab02-reflection.md` at the root of `lab-files/lab-02/`.

Plus, if you have time: a protected nested route with a streaming layout (in either framework or both).

Time budget

Phase	Minutes
Setup — three things running at once	10
Part A — Implement in React Router v7	55
Part B — Port to Next.js App Router	30
Part C — Reflection	10
Stretch (if time): nested streaming route	+

If you finish early, do the stretch task or help a neighbor.

Setup

You'll be running three things simultaneously: the shared Express backend, an RR v7 client, and a Next.js client. Each goes in its own terminal.

Terminal 1 — backend

```
cd lab-files/lab-02/social-media/server
npm install
npm run dev
```

The server listens on `http://localhost:4000`. It exposes `/api/login`, `/api/signup`, `/api/me`, `/api/posts`, and a few others. CORS is configured to accept both Vite (5173) and Next.js (3000) origins.

Terminal 2 — RR v7 client

```
cd lab-files/lab-02/social-media-rr-v7
npm install
npm run dev
```

You should see Vite log a URL around `http://localhost:5173`. The starter has a placeholder / route; you'll add the real routes during Part A.

Terminal 3 — Next.js client

```
cd lab-files/lab-02/social-media-nextjs
npm install
npm run dev
```

You should see Next.js log `http://localhost:3000`. The starter has the App Router scaffolding and an empty home page; you'll fill it out during Part B.

Verify the backend works

In a fourth terminal (or your favorite HTTP client):

```
curl http://localhost:4000/api/setup-check
# should return {"ok": true, "message": "social-media backend ready", ...}
```

If that works, you're set. **Make a backup copy of lab-files/lab-02/ before you start changing things** — that way you can recover if anything goes sideways:

```
cp -R lab-files/lab-02 lab-files/lab-02-backup
```

Windows PowerShell:

```
Copy-Item -Recurse lab-files/lab-02 lab-files/lab-02-backup
```

Part A — React Router v7 (55 min)

You're implementing four routes:

Route	Behavior
/login	Email + password form. POSTs to /api/login. On success, sets the session cookie and redirects to /home.
/signup	Name + email + password form. POSTs to /api/signup. On success, redirects to /login.
/logout	Clears the session cookie and redirects to /login.
/home	Protected. Loads posts from /api/posts and renders them. Redirects to /login?next=/home if not authenticated.

Step 1 — Wire up route configuration

Open `app/routes.js`. Add the four routes. Use the `route()` and `index()` helpers from `@react-router/dev/routes`. The /home route should be wrapped in a layout route that handles the auth redirect.

Step 2 — Implement /login and /signup

Each route file exports an action (handles the form POST) and a default component (renders the form). Use `<Form method="post">` from `react-router`. On success in the action, return a `redirect()` to the next route.

Hint: forward the backend's `Set-Cookie` header on the redirect response so the browser stores the session cookie. The starter has a `api.server.js` helper to make this easier — use it.

Step 3 — Implement /logout

A POST-only route. The action clears the session cookie and redirects to `/login`. The component can be a simple “Sign out” button inside a form.

Step 4 — Implement /home with auth redirect

The route's loader should:

- Call `/api/me` with the inbound cookie forwarded.
- If it 401s, throw `redirect('/login?next=/home')`.
- Otherwise, fetch posts from the backend and return them.

The component reads the loader data with `useLoaderData()` and renders the post list.

Step 5 — Test the full flow

- Hit `/home` in a fresh browser. You should be redirected to `/login?next=/home`.
- Sign up with a fake email. You should land on `/login`.
- Log in. You should land on `/home` and see posts.
- Sign out. You should land on `/login`.

When all four routes work end-to-end, you're done with Part A.

Part B — Next.js App Router (30 min)

Port the same feature set to Next.js. The biggest structural difference: auth happens in **middleware**, not in route loaders.

Step 1 — Wire up the App Router

Create these files (some may already exist as placeholders):

```
app/
├── (auth)/
│   ├── login/page.jsx
│   └── signup/page.jsx
└── home/page.jsx
```

```
└─ logout/route.js
└─ middleware.js (at the project root, not in app/)
```

The (auth) route group keeps /login and /signup URLs at the top level while letting you share an auth layout if you want.

Step 2 — Implement the middleware

middleware.js runs before every matching request. Check for the session cookie. If missing, redirect to /login. Use a matcher config to limit it to protected routes.

```
export const config = { matcher: ['/home/:path*'] };
```

Step 3 — Implement /login and /signup

Each page.jsx is a Server Component that renders a form. The form's action is a Server Action (a function with 'use server') that calls the backend, sets the session cookie, and redirects.

For setting cookies from a Server Action, use:

```
import { cookies } from 'next/headers';
cookies().set('session', token, { httpOnly: true, secure: true, path: '/' });
```

Step 4 — Implement /logout

A route.js exporting a POST handler. Clear the cookie, redirect to /login. (You can also do it as a Server Action inside a form — both work.)

Step 5 — Implement /home

A Server Component that fetches posts from the backend (you can await fetch() directly — Server Components support that natively) and renders them. The middleware handles the auth redirect, so this component doesn't need to check.

Step 6 — Test the full flow

Same as Part A — sign up, log in, see posts, log out.

Part C — Reflection (10 min)

Create lab02-reflection.md at the root of lab-files/lab-02/. Answer in one paragraph each:

1. **Which framework felt more natural for this task?** Why? Be specific — name a file, a concept, a trade-off.
2. **Where did the two diverge structurally?** What did each force you to do that the other didn't?
3. **If your team had to pick one tomorrow, which would you advocate for, and why?**

Stretch — protected nested route with streaming layout

Pick one of the two clients and add a `/home/profile` route that:

1. Lives under the same auth-protected umbrella as `/home`.
2. Uses a layout that streams: the page shell renders immediately, the data-heavy parts of the profile arrive in their own `Suspense` boundaries.
3. Shows a nice loading skeleton (not a spinner — a skeleton).

In RR v7, that's nested route configuration + `<Suspense>` + `defer()` in the loader.

In Next.js, that's a `loading.jsx` for the segment + `<Suspense>` boundaries inside the page.

Hints if you get stuck

Both frameworks

- **CORS rejected by the backend:** confirm both 5173 and 3000 are in the server's allowed origins list. The shared backend is set up for both.
- **Session cookie not set:** check that the response sets the cookie with `Path=/`, `HttpOnly`, and (in dev) without `Secure`. Browser dev tools → Application → Cookies will show you what was sent.
- **/api/me returns 401 even after login:** the cookie isn't being sent on subsequent requests. Make sure your fetch calls include `credentials: 'include'` (RR v7) or that you're using the cookie-aware `fetch` (Next.js).

React Router v7

- **useLoaderData returns null:** the loader didn't run or the route isn't wired into `routes.js` properly. Check the routes file and the file naming.
- **<Form> posts but action doesn't run:** import `Form` from `react-router`, not from anywhere else. Plain `<form>` would just do a regular browser POST.
- **Cookie set in action isn't readable in loader on the same redirect:** redirects in RR v7 follow the `Set-Cookie` header. Check that you're returning the redirect from the action, not the component, and that the headers are merged.

Next.js

- **Middleware redirects in a loop:** your matcher is too broad. `/home/:path*` matches `/home` and below. Don't put `/login` or `/signup` under it, or they'll redirect themselves to themselves.
- **Server Action gets a "cannot find module" error:** `'use server'` files must export only async functions and only Server-Component-compatible imports. Don't import a client-only library at the top.
- **cookies().set() errors with "must be in a Server Action or Route Handler":** you tried to set a cookie from a Server Component. Move the set into the action.

Reference solutions

After the lab:

- [solutions/lab-02-rr-v7/](#) — RR v7 reference
- [solutions/lab-02-nextjs/](#) — Next.js reference

Don't peek before you've finished your own implementation — the value of the lab is in the comparison, and you can't compare what you haven't built.

Lab 3 — Pick-Your-Tool State Refactor

Time: 90 minutes

Format: Hands-on. Work alone.

Goal: Take the social-media app — which has prop-drilling, an over-stuffed `AppContext`, and ad-hoc context for half a dozen unrelated concerns — and refactor its client state using one tool of your choice. Then defend your choice in 60 seconds.

The point of this lab isn't to learn one of three tools deeply. It's to make a real engineering decision and live with the consequences.

What you'll have at the end

1. A refactored social-media client where the prop-drilling is gone, the bloated context is gone, and client state lives in a coherent place.
2. A short written justification (3–5 bullets) for why you picked the tool you picked, saved as `lab03-justification.md` at the root of `lab-files/lab-03/`.
3. A 60-second pitch you'll give to the class.

Plus, if you have time: a port to a second tool and a written comparison.

Time budget

Phase	Minutes
Setup + survey the existing state	15
Pick your tool + sketch a plan	10
Refactor	50
Write justification + prepare pitch	15
Stretch (if time): port to a second tool	+

If you finish early, do the stretch task.

Setup

You're working in `lab-files/lab-03/`. The starter is the post-Lab-2 state of `social-media-rr-v7/` PLUS a deliberately bloated `AppContext` that you'll be refactoring.

Terminal 1 – backend

```
cd lab-files/lab-03/social-media/server
npm install
npm run dev
```

Terminal 2 – client

```
cd lab-files/lab-03/social-media-rr-v7
npm install
npm run dev
```

Visit `http://localhost:5173`. Sign up and log in. The app should show a posts feed and a sidebar.

Survey what's there

Spend ten minutes reading. Don't write any code yet. Open these files and just look:

- `app/routes/home.jsx` — wraps everything in `<AppProvider>`. Notice how many things it provides.
- `app/context/AppContext.jsx` — the bloated context. Count the values it exposes.
- `app/components/PostsFeed.jsx` — see how many props it takes that it doesn't itself use (it forwards them to children).
- `app/components/Sidebar.jsx` — uses `useAppContext()` to read three different values, even though it doesn't need most of the context.

Write down on a sticky note (or in a comment in your editor):

- Three values that are clearly **server state** (and should leave the client store entirely)
- Three values that are clearly **client state shared widely** (good candidates for the new store)
- Three values that are **local component state** (and shouldn't be in any store at all)

This list is your refactor plan.

Make a backup

```
cp -R lab-files/lab-03 lab-files/lab-03-backup
```

Windows PowerShell:

```
Copy-Item -Recurse lab-files/lab-03 lab-files/lab-03-backup
```

Pick your tool (10 min)

Re-read Module 4's "decision framework — one more time" slide (the five-question framework) and apply it to this app. Then pick **one** tool:

- **Zustand** — minimal API, hook-shaped, no Provider needed. Default for "I just want a small store."
- **Jotai** — atoms compose into a graph. Strong fit if your derived state is itself complex.
- **Redux Toolkit + RTK Query** — strongest conventions, best DevTools. Pick if you want the structure for a large team or want server state in the same toolset.

Write your three-to-five-bullet justification draft now, while the framework is fresh. You'll polish it at the end. The bullets should answer:

1. Why this tool fits this app's state shape.
2. What you considered but rejected, and why.
3. What you'd lose if you switched to a different tool.

If you can't write the bullets, you don't have a real justification yet. Pick again.

Refactor (50 min)

There's no single right way to do this — that's the point. But here's a sensible order regardless of tool:

Step 1 — Strip the bloat from the existing context

Whatever stays in context should be **just** plumbing — auth user, theme, query client. Move everything else somewhere appropriate. Don't worry about breaking things on this pass; the tests will surface what's broken.

Step 2 — Move server state OUT entirely

Anything you flagged as server state goes through `fetch` (or a route loader) for now — Lab 4 introduces TanStack Query for these. Don't put server data in your new store.

Step 3 — Create your store

Whatever shape your tool wants. Aim for:

- Slices grouped by feature, not by data shape (e.g., `useDraftsStore`, not `useStringsStore`)
- Actions named for what they do, not for what they set (`removePost`, not `setPosts`)
- Selectors that return the smallest shape consumers need

Step 4 — Replace `useAppContext()` with store hooks

In each consuming component, swap context reads for store reads. Use selectors to read just the slice you need.

Step 5 — Push prop-drilled values into the store

The components that currently take a `currentUser` prop just to forward it three layers down are the easy wins. Read from the store at the leaf component instead.

Step 6 — Run the app

Click around. Sign in, post, edit, switch theme, open the sidebar. Watch for re-render storms in the React DevTools profiler — if a single setter is causing a whole feature to re-render, your selectors are too broad.

Write justification + prepare pitch (15 min)

The justification

Polish the three-to-five bullets you started with. Each bullet is one tight sentence. Save as `lab03-justification.md` at the root of `lab-files/lab-03/`.

A good justification cites specifics from the codebase. “Zustand fit because most of this app’s client state is flat — a single sidebar flag, a draft, a theme — and the per-slice selectors keep the post-list re-rendering only when the post list changes” beats “Zustand is simple and small.”

The 60-second pitch

You’ll have one minute when the class reconvenes. Plan three beats:

1. **What I picked** — one sentence
2. **Why it fit this app** — one or two sentences
3. **What I’d watch out for** — one sentence

That’s it. No live demos, no slides — just talking. Practice it once before class returns.

Stretch — port to a second tool

If you have time, port your refactor to a second tool of your choice. Make a copy first so you don’t lose your first refactor:

```
cp -R lab-files/lab-03/social-media-rr-v7 lab-files/lab-03/social-media-rr-v7-second-tool
```

When you’re done, write a `lab03-comparison.md` answering:

1. Which tool fit this codebase better, and why?
2. What was the hardest thing to express in each?
3. Which would you advocate for at work tomorrow?

The comparison is more valuable than the second refactor. Be specific — cite files, cite lines.

Hints by tool

Zustand

- **A common mistake:** importing the entire store into every component. Pass a selector instead: `useStore((s) => s.draft)` re-renders only when draft changes.
- **Where to put the store file:** `app/stores/<feature>.js`. Don't dump everything into one giant store unless it's genuinely one feature.
- **DevTools:** `import { devtools } from 'zustand/middleware'`, wrap your store creator. Action names show up in Redux DevTools.
- **Persist a slice (e.g., theme) to localStorage:** the persist middleware. Don't roll your own.

Jotai

- **A common mistake:** putting a whole feature into one big atom. The win is fine-grained subscriptions; one big atom defeats the purpose.
- **Derived atoms:** `atom((get) => get(itemsAtom).length)` — use them for anything you'd otherwise compute in render.
- **Async atoms:** `atom(async (get) => fetch('/api/...'))` integrates with Suspense. Useful for the server-state pieces if you don't want TanStack Query yet.
- **Provider needed only if** you want isolated atom scopes (e.g., per-route). Most apps don't need one.

Redux Toolkit + RTK Query

- **A common mistake:** writing manual reducers. Use `createSlice` — it gives you actions and reducers in one shot.
 - **Don't write a setX reducer for every field.** Reducers should describe transitions (`addItem`, `removeItem`, `clearAll`), not setters.
 - **Selectors with `createSelector`:** memoized derivations. Keeps components from re-rendering on unrelated state changes.
 - **RTK Query for server state:** if you go this route, use `createApi` for `/api/posts`, `/api/me`, etc. — not `useEffect` + `fetch`. Lab 4 will revisit.
-

Reference solutions

After the lab, three reference solutions are available:

- [solutions/lab-03-zustand/](#)
- [solutions/lab-03-jotai/](#)
- [solutions/lab-03-rtk/](#)

Compare against the one you chose. Then read at least one of the others — the comparison is half the lesson.

Lab 4 — TanStack Query Migration

Time: 75 minutes

Format: Hands-on. Work alone.

Goal: Take the social-media app’s loader-based fetching and migrate the home feed to TanStack Query. Add an optimistic update for the create-post flow. Wrap loading and error UX in `Suspense` and an `ErrorBoundary`.

This lab is the practical follow-up to Module 5. By the end, your data layer should look like the canonical 2026 pattern.

What you’ll have at the end

1. A social-media client where the home page’s posts go through `useSuspenseQuery` and the create-post flow goes through `useMutation`.
2. A create-post flow with optimistic updates — new posts appear instantly, roll back if the server rejects.
3. A posts feed wrapped in `<Suspense>` (with a skeleton fallback) and an `<ErrorBoundary>` (with a retry button).

Plus, if you have time: an infinite-scroll posts feed using `useInfiniteQuery`.

Time budget

Phase	Minutes
Setup + survey the existing fetch code	10
Part A — Wire up TanStack Query	10
Part B — Migrate reads to <code>useSuspenseQuery</code>	20
Part C — Migrate writes to <code>useMutation</code>	15
Part D — Add optimistic create-post	10
Part E — <code>Suspense</code> + <code>ErrorBoundary</code>	10
Stretch (if time): infinite scroll	+

If you finish early, do the stretch task.

Setup

You're working in `lab-files/lab-04/`. The starter is the post-Lab-3 state of `social-media-rr-v7/` (Zustand for client state) plus the shared backend.

Terminal 1 – backend

```
cd lab-files/lab-04/social-media/server
npm install
npm run dev
```

Terminal 2 – client

```
cd lab-files/lab-04/social-media-rr-v7
npm install
npm run dev
```

Visit `http://localhost:5173`. Sign up and log in. The home feed should show.

Survey what's there

Spend ten minutes reading the data layer. Open these files:

- `app/lib/api.js` — the existing fetch wrapper (client-side)
- `app/lib/api.server.js` — the loader-side wrapper that forwards cookies
- `app/routes/home.jsx` — the loader fetches posts on the server; client renders directly from `useLoaderData()`

Note the smells:

- The home page can't refetch on focus — only on full navigation
- A new post requires a manual refetch path
- Errors render as inline strings
- Nothing dedups simultaneous requests

These are the things TanStack Query is going to fix.

Make a backup

```
cp -R lab-files/lab-04 lab-files/lab-04-backup
```

Windows PowerShell:

```
Copy-Item -Recurse lab-files/lab-04 lab-files/lab-04-backup
```

Install TanStack Query

```
cd lab-files/lab-04/social-media-rr-v7
npm install @tanstack/react-query@5
npm install @tanstack/react-query-devtools react-error-boundary
```

(The `@5` pin matters — TanStack Query's API changed significantly between v4 and v5.)

Part A — Wire up TanStack Query (10 min)

Step 1 — Create the QueryClient

In `app/lib/queryClient.js`:

```
import { QueryClient } from '@tanstack/react-query';

export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 30 * 1000,
      retry: 2,
      refetchOnWindowFocus: true,
    },
    mutations: {
      retry: false,
    },
  },
});
```

Step 2 — Wrap the home subtree

Wrap the home route's body (or its outermost component) with `<QueryClientProvider>` from `@tanstack/react-query`. Add the `<ReactQueryDevtools />` panel — you'll use it constantly.

Step 3 — Set up a query-keys factory

In `app/lib/queryKeys.js`:

```
export const queryKeys = {
  posts: {
    all: ['posts'],
    list: (filters = {}) => ['posts', 'list', filters],
    detail: (id) => ['posts', 'detail', id],
  },
  user: {
    me: ['user', 'me'],
  },
};
```

This isn't required, but it pays for itself within an hour. Do it now.

Part B — Migrate reads to useSuspenseQuery (20 min)

Step 1 — The posts list

The home page currently gets posts from the loader. Move that fetch to the client via useSuspenseQuery:

```
import { useSuspenseQuery } from '@tanstack/react-query';
```

```
const { data: posts } = useSuspenseQuery({
  queryKey: queryKeys.posts.list(),
  queryFn: () => api.listPosts(),
});
```

The home loader should still do the auth gate (call /api/me, redirect on 401) but no longer fetch posts.

Step 2 — The current user

Replace any direct /api/me calls in components with a useQuery-wrapped hook:

```
const { data: user } = useQuery({
  queryKey: queryKeys.user.me,
  queryFn: () => api.getMe(),
  staleTime: 5 * 60 * 1000, // 5 min — user info changes rarely
});
```

(Plain useQuery, not Suspense — the header should be allowed to briefly show “guest” while the user hydrates.)

Step 3 — Verify

Run the app. Click around. Open the Devtools panel — you should see ['posts', 'list', {}] and ['user', 'me'] cached. Navigate between routes — re-visiting no longer hits the network (cache is fresh).

Part C — Migrate writes to useMutation (15 min)

Step 1 — Create-post

Add a <NewPostForm /> component that uses useMutation:

```
const queryClient = useQueryClient();

const { mutate, isPending, error } = useMutation({
  mutationFn: (newPost) => api.createPost(newPost),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: queryKeys.posts.all });
  },
});
```

```
});

function handleSubmit(e) {
  e.preventDefault();
  mutate({ title, body });
}
```

The form's button should disable while `isPending` is true, and show the error if the mutation failed.

Step 2 — Verify

Create a new post. Watch the Devtools — you should see the mutation fire, succeed, and the `['posts', 'list', {}]` query refetch in the background. The new post appears in the feed.

Part D — Add optimistic create-post (10 min)

The current create-post flow has a noticeable gap between submission and the post appearing. Optimistic updates close that gap.

Update the `useMutation` from Part C:

```
const { mutate } = useMutation({
  mutationFn: (newPost) => api.createPost(newPost),

  onMutate: async (newPost) => {
    await queryClient.cancelQueries({ queryKey: queryKeys.posts.list() });
    const snapshot = queryClient.getQueryData(queryKeys.posts.list());
    queryClient.setQueryData(queryKeys.posts.list(), (old = []) => [
      { ...newPost, id: `optimistic-${Date.now()}`, sending: true },
      ...old,
    ]);
    return { snapshot };
  },

  onError: (err, newPost, ctx) => {
    queryClient.setQueryData(queryKeys.posts.list(), ctx.snapshot);
  },

  onSettled: () => {
    queryClient.invalidateQueries({ queryKey: queryKeys.posts.list() });
  },
});
```

In your posts-feed component, render posts with the sending flag styled differently — a CSS class for `.sending { opacity: 0.55; font-style: italic; }` works.

Verify

Submit a new post. The new post should appear at the top of the feed instantly, dimmed. After the server confirms, it un-dims (the invalidation triggers a refetch and replaces the optimistic stub with the real post).

To test the rollback: temporarily make the server return 500 (post with the body boom — the lab backend treats that as a magic error trigger). The optimistic post should disappear when the error settles.

Part E — Suspense + ErrorBoundary (10 min)

Switch the feed from status flags to Suspense mode.

Step 1 — Use the Suspense variant

Confirm your posts hook uses `useSuspenseQuery`. The component body no longer needs `isPending` or error checks — boundaries handle them.

Step 2 — Wrap with Suspense and ErrorBoundary

```
import { Suspense } from 'react';
import { ErrorBoundary } from 'react-error-boundary';
import { QueryErrorResetBoundary } from '@tanstack/react-query';

function HomePage() {
  return (
    <QueryErrorResetBoundary>
      ({ { reset } }) => (
        <ErrorBoundary
          onReset={reset}
          fallbackRender={({ error, resetErrorBoundary }) => (
            <ErrorPanel error={error} onRetry={resetErrorBoundary} />
          )}
        >
          <Suspense fallback={<PostsFeedSkeleton />}>
            <Feed />
          </Suspense>
        </ErrorBoundary>
      )}
    </QueryErrorResetBoundary>
  );
}
```

Feed is the component that calls `useSuspenseQuery`. `<PostsFeedSkeleton />` is a CSS-only loading skeleton — three or four blurred placeholder cards. `<ErrorPanel />` shows the error and a retry button.

Step 3 — Verify

Hard-refresh the page. You should see the skeleton briefly, then the real feed. To test the error boundary: temporarily break the API URL in `api.js`. The skeleton should appear, then the `ErrorPanel` with a working “Try again” button.

Stretch — infinite-scroll feed

Use `useInfiniteQuery` to add pagination-by-scroll to the posts feed.

```
const {
  data,
  fetchNextPage,
  hasNextPage,
  isFetchingNextPage,
} = useInfiniteQuery({
  queryKey: queryKeys.posts.list({ infinite: true }),
  queryFn: ({ pageParam = 0 }) => api.listPosts({ cursor: pageParam }),
  getNextPageParam: (lastPage) => lastPage.nextCursor ?? undefined,
  initialPageParam: 0,
});
```

```
const posts = data?.pages.flatMap((p) => p.items) ?? [];
```

Render the posts as usual, then add an intersection-observer-triggered `fetchNextPage()` at the bottom of the feed. (The `react-intersection-observer` library makes this easy if you don't want to roll your own.)

The backend supports cursor-based pagination via `?cursor=` and `?limit=` once you opt in — see `solutions/lab-04-stretch/` for the backend changes if you go this route.

Hints if you get stuck

- **useQuery returns undefined data forever:** the `queryFn` isn't returning a promise that resolves. Make sure you return `r.json()` rather than just `r.json()`.
- **DevTools shows the mutation succeeding but the feed isn't updating:** the invalidation key doesn't match the query key. `queryKeys.posts.all` should be a prefix of `queryKeys.posts.list()`. Check the actual keys in DevTools.
- **Optimistic post appears but never goes away:** your `onSettled` is missing or invalidating the wrong key. Without invalidation, the optimistic stub stays forever.
- **useSuspenseQuery errors with “no QueryClient”:** the component using it is rendered outside the `QueryClientProvider`. Move the provider higher in the tree.
- **<Suspense> shows the fallback forever:** something is recreating the `queryFn`'s promise on every render. Make sure your `queryFn` is stable (defined outside the component, or memoized via `useCallback` if it depends on props).

- **Server returns 401 after login:** TanStack Query doesn't carry cookies by default if your fetch wrapper doesn't include credentials: 'include'. Update your `api.js` wrapper.
-

Reference solutions

- `solutions/lab-04-tanstack-query/` — main solution
- `solutions/lab-04-stretch/` — infinite scroll

Lab 5 — Server Components Dashboard

Time: 75 minutes

Format: Hands-on. Work alone.

Goal: Build a small project-tracker dashboard using Server Components for data and a Server Action for the create-task form. Document each client/server boundary decision and explain why each 'use client' was necessary.

What you'll have at the end

1. A dashboard page (`/dashboard`) showing:
 - Three stats cards (open tasks, in-progress tasks, done tasks) — Server Components
 - A recent-activity feed — Server Component
 - A team members list — Server Component
 - An “Add task” form — Server Action with `useActionState` for validation
 - A filter widget that toggles which task statuses appear in the feed — the only Client Component
2. A `BOUNDARY-NOTES.md` at the root of `lab-files/lab-05/` explaining each 'use client' you used and why each one was necessary.

Plus, if you have time: streaming with multiple `Suspense` boundaries so the stats cards appear instantly and the slower components (activity, team) stream in.

Time budget

Phase	Minutes
Setup + read the starter	10
Part A — Add stats cards	15
Part B — Add activity feed + team list	15
Part C — Add task with Server Action	25

Part D — Filter widget (the Client Component) 10

Stretch (if time): streaming with Suspense +

If you finish early, do the stretch task.

Setup

You're working in `lab-files/lab-05/`. The starter is a Next.js 15 project that ships with mock data and minimal scaffolding. Spend ten minutes reading the starter before writing any code.

```
cd lab-files/lab-05/server-components-dashboard
npm install
npm run dev
```

Open `http://localhost:3000`. You'll see a placeholder dashboard with one stub card.

Read these files

- `app/dashboard/page.jsx` — the dashboard page (currently a stub Server Component)
- `app/dashboard/layout.jsx` — shared chrome
- `lib/data.js` — mock data layer with `getTaskCounts()`, `getRecentActivity()`, `getTeamMembers()`, `getTasks()`, `createTask()`. Each function is async and has a deliberate `await sleep(...)` to make the streaming demo meaningful.
- `app/dashboard/StubCard.jsx` — example of a server-side card you can copy from
- `app/dashboard/FilterToggle.jsx` — empty Client Component placeholder

Make a backup

```
cp -R lab-files/lab-05 lab-files/lab-05-backup
```

Windows PowerShell:

```
Copy-Item -Recurse lab-files/lab-05 lab-files/lab-05-backup
```

Boundary diary

Create `lab-files/lab-05/BOUNDARY-NOTES.md`:

```
# Boundary decisions
```

For each ``'use client`` I added, this is what made it necessary.

You'll be appending to this file as you go.

Part A — Add stats cards (15 min)

Build three Server Components that show task counts: OpenTasksCard, InProgressCard, DoneCard.

Step 1 — Make a StatCard Server Component

In `app/dashboard/StatCard.jsx`:

```
export default function StatCard({ label, value, accent }) {  
  return (  
    <article className={`stat-card ${accent}`}>  
      <h3>{label}</h3>  
      <p className="stat-value">{value}</p>  
    </article>  
  );  
}
```

Note: no 'use client' directive. This is a Server Component.

Step 2 — Use it in page.jsx

```
import { getTaskCounts } from '@/lib/data';  
import StatCard from './StatCard';  
  
export default async function Dashboard() {  
  const counts = await getTaskCounts();  
  
  return (  
    <main>  
      <h1>Project tracker</h1>  
      <section className="stats-row">  
        <StatCard label="Open" value={counts.open} accent="blue" />  
        <StatCard label="In progress" value={counts.inProgress} />  
        <StatCard label="Done" value={counts.done} accent="green" />  
      </section>  
    </main>  
  );  
}
```

Step 3 — Verify

Run the app. You should see three cards with real numbers. Open DevTools → Network tab → the page request. Look at the response: there's no JS for the card components. The numbers are baked into the HTML.

This is the headline win of Server Components — the cards ship as zero JS.

Part B — Add activity feed and team list (15 min)

Step 1 — Activity feed

In app/dashboard/ActivityFeed.jsx:

```
import { getRecentActivity } from '@lib/data';

export default async function ActivityFeed() {
  const events = await getRecentActivity();
  return (
    <section className="activity">
      <h2>Recent activity</h2>
      <ul>
        {events.map((e) => (
          <li key={e.id}>
            <strong>{e.actor}</strong> {e.verb} <em>{e.target}</em>
            <span className="time"> · {e.timeAgo}</span>
          </li>
        ))}
      </ul>
    </section>
  );
}
```

Step 2 — Team list

In app/dashboard/TeamList.jsx:

```
import { getTeamMembers } from '@lib/data';

export default async function TeamList() {
  const members = await getTeamMembers();
  return (
    <section className="team">
      <h2>Team</h2>
      <ul>
        {members.map((m) => (
          <li key={m.id}>
            <span>{m.name}</span>
            <span className="role">{m.role}</span>
          </li>
        ))}
      </ul>
    </section>
  );
}
```

Step 3 — Compose them in page.jsx

```
import ActivityFeed from './ActivityFeed';
import TeamList from './TeamList';
```

```

export default async function Dashboard() {
  const counts = await getTaskCounts();
  return (
    <main>
      {/* stats row from Part A */}
      <div className="dashboard-grid">
        <ActivityFeed />
        <TeamList />
      </div>
    </main>
  );
}

```

Step 4 — Verify

The page now waits for the slowest of `getTaskCounts`, `getRecentActivity`, and `getTeamMembers` before rendering anything. You can see the wait in the dev server logs (each function logs its sleep). Don't fix that yet — the streaming stretch is what fixes it.

Part C — Add task with Server Action (25 min)

This is the heart of the lab. You'll add a form that creates a task via a Server Action, with `useActionState` handling validation and the pending state.

Step 1 — Create the action

In `app/actions/tasks.js`:

```

'use server';

import { createTask } from '@lib/data';
import { revalidatePath } from 'next/cache';

export async function addTaskAction(prevState, formData) {
  const title = formData.get('title')?.toString().trim();
  const status = formData.get('status')?.toString();

  if (!title) {
    return { error: 'Title is required.', title, status };
  }
  if (title.length > 100) {
    return { error: 'Title must be 100 characters or less.', title, status };
  }
  if (!['open', 'in-progress', 'done'].includes(status)) {
    return { error: 'Invalid status.', title, status };
  }
}

```

```

    await createTask({ title, status });
    revalidatePath('/dashboard');
    return { success: true };
}

```

The `revalidatePath` call invalidates the dashboard's cached data so the new task shows up.

Step 2 — Build the form (Client Component)

In `app/dashboard/AddTaskForm.jsx`:

```

'use client';

import { useActionState } from 'react';
import { addTaskAction } from '@app/actions/tasks';
import SubmitButton from './SubmitButton';

const initialState = { error: null, title: '', status: 'open', success: false };

export default function AddTaskForm() {
  const [state, action] = useActionState(addTaskAction, initialState);

  return (
    <form action={action} className="add-task-form">
      <h3>Add a task</h3>
      <input
        name="title"
        defaultValue={state.title}
        placeholder="What needs doing?"
        autoFocus
        required
      />
      <select name="status" defaultValue={state.status}>
        <option value="open">Open</option>
        <option value="in-progress">In progress</option>
        <option value="done">Done</option>
      </select>
      {state.error && <p className="error">{state.error}</p>}
      {state.success && <p className="success">Task added.</p>}
      <SubmitButton />
    </form>
  );
}

```

Step 3 — Build a reusable submit button

In `app/dashboard/SubmitButton.jsx`:

```
'use client';

import { useFormStatus } from 'react-dom';

export default function SubmitButton() {
  const { pending } = useFormStatus();
  return (
    <button type="submit" disabled={pending}>
      {pending ? 'Adding...' : 'Add task'}
    </button>
  );
}
```

Step 4 — Add the form to the dashboard

```
import AddTaskForm from './AddTaskForm';
```

```
// inside Dashboard's return...
<AddTaskForm />
```

Step 5 — Verify

Try the full flow:

1. Submit with no title → error message appears, no task created
2. Submit with a 200-character title → length error
3. Submit valid title → success message, stats card updates after revalidatePath

Try without JavaScript: open DevTools → Settings → Disable JavaScript, then submit the form. It should still work because Server Actions support progressive enhancement. Re-enable JS afterward.

Part D — Filter widget + Boundary documentation (10 min)

The activity feed could use a “show only completed” toggle. This is the only piece of UI in the dashboard that genuinely needs to be a Client Component — it has interactive client state.

Step 1 — Build the toggle

In `app/dashboard/FilterToggle.jsx`:

```
'use client';

import { useState } from 'react';

export default function FilterToggle({ children }) {
  const [completedOnly, setCompletedOnly] = useState(false);
```

```

return (
  <div>
    <label className="filter">
      <input
        type="checkbox"
        checked={completedOnly}
        onChange={(e) => setCompletedOnly(e.target.checked)}
      />
      Show completed only
    </label>
    {children}
  </div>
);
}

```

Step 2 — Use it correctly

The trick: FilterToggle is a Client Component, but its children can still be the Server Component `<ActivityFeed />`. That's the composition pattern from Module 6's "pass Client Components as children" slide.

```

import FilterToggle from './FilterToggle';
import ActivityFeed from './ActivityFeed';

```

```

// inside Dashboard's return...
<FilterToggle>
  <ActivityFeed />
</FilterToggle>

```

Step 3 — Document boundary decisions

Open `lab-files/lab-05/BOUNDARY-NOTES.md` and write a short paragraph for each component that uses `'use client'`. Cite the exact reason. Examples:

Boundary decisions

AddTaskForm.jsx — `'use client'`

Needed because: uses `'useActionState'`. Must be a Client Component to wire the form's pending state and the validation feedback to the rendered DOM.

SubmitButton.jsx — `'use client'`

Needed because: uses `'useFormStatus'`, which only works inside a Client Component descendant of a `<form>`. Extracted into its own component so the rest of the form's structure could stay declarative.

FilterToggle.jsx — `'use client'`

Needed because: uses `useState` for the checkbox's local UI state. The children prop receives a Server Component (`<ActivityFeed />`) – that part stays server-rendered.

Components I deliberately did NOT mark `'use client'`

- `StatCard`, `ActivityFeed`, `TeamList`, the dashboard page itself: pure server-rendered, no state, no event handlers, can read the data layer directly.
- The `addTaskAction` itself uses the `'use server'` directive, not `'use client'` – it runs server-side when called from the form.

Stretch — streaming with multiple Suspense boundaries

Right now the page waits for ALL data before rendering anything. With multiple Suspense boundaries, the shell ships immediately, and each section streams in independently.

Step 1 — Wrap each slow section in Suspense

```
import { Suspense } from 'react';

export default function Dashboard() {
  return (
    <main>
      <h1>Project tracker</h1>

      <Suspense fallback={<StatsRowSkeleton />}>
        <StatsRow />
      </Suspense>

      <div className="dashboard-grid">
        <Suspense fallback={<ActivityFeedSkeleton />}>
          <FilterToggle>
            <ActivityFeed />
          </FilterToggle>
        </Suspense>

        <Suspense fallback={<TeamListSkeleton />}>
          <TeamList />
        </Suspense>
      </div>

      <AddTaskForm />
    </main>
  );
}
```

Step 2 — Extract the awaiting work into child Server Components

Now Dashboard itself doesn't await anything. The `await` lives in `<StatsRow />`, `<ActivityFeed />`, etc. Each one suspends inside its own boundary.

Step 3 — Build the skeleton components

Three small Server Components that render the same shape as their real counterparts but with placeholder text. CSS placeholder shimmer works.

Step 4 — Verify

Open the page. You should see:

1. The page shell + skeletons appear instantly
2. The stats row fills in first (fastest query)
3. The activity feed and team list each fill in as their queries resolve

The Network tab shows the response streaming in chunks. If you watch the dev server logs, the three queries fire concurrently rather than serially.

Hints if you get stuck

- **“Cannot use `useState` in a Server Component” error:** A component you wrote uses `useState` but doesn't have `'use client'` at the top. Add it, or refactor so the state lives in a Client Component child.
- **“Cannot pass a function as a prop from Server to Client” error:** You passed an inline event handler from a Server Component to a Client Component. Either move the handler into the Client Component or convert the function to a Server Action with `'use server'`.
- **Form submits but the dashboard doesn't update:** missing `revalidatePath` (or you're on RR v7, in which case the equivalent is the route's loader rerunning automatically after the action).
- **`useFormStatus` returns `null`:** the component using it is rendered outside a `<form>`. It must be a descendant of a form element, not the form itself.
- **Suspense fallback never disappears:** something in the suspending child is creating a new promise on every render. Move the data fetch into a Server Component that awaits it directly.
- **“Module not found: 'fs'” or ‘crypto’:** a server-only module was imported into a Client Component. Add a `'server-only'` import to the offending file to get a clearer error.

Reference solutions

- `solutions/lab-05-dashboard/` — main solution
- `solutions/lab-05-streaming/` — streaming stretch

Lab 6 — Profile and Optimize

Time: 75 minutes

Format: Hands-on. Work alone.

Goal: Profile the social-media app, identify the top three bottlenecks, apply at least three optimization techniques, and report before/after numbers. No vibes — actual measurements.

The point of this lab is to build the measurement-first habit. By the end, you should be able to point at a number that proves your fix worked.

What you'll have at the end

1. A `lab06-baseline.md` file with your starting numbers — Profiler timings for the top three components, Lighthouse scores, bundle size.
2. Three (or more) sets of changes, each implementing one optimization with before/after numbers documented.
3. A `lab06-results.md` summary tying it all together — what you changed, what improved, what didn't.

Both `.md` files go at the root of `lab-files/lab-06/`.

Plus, if you have time: a virtualized posts feed using TanStack Virtual.

Time budget

Phase	Minutes
Baseline measurement	15
Optimization 1	15
Optimization 2	15
Optimization 3	15
Write up <code>lab06-results.md</code>	15
Stretch (if time): virtualization	+

If you finish early, do the stretch task or pick a fourth optimization.

Setup

You're working in `lab-files/lab-06/`. The starter is the post-Lab-4 state of `social-media-rr-v7/` (TanStack Query is in place) PLUS a set of deliberately seeded performance problems: a search input that re-renders the entire feed on every keystroke, a

TrendingSidebar that recomputes on every render, and a Composer that eagerly imports a “heavy” module.

Terminal 1 – backend (the Lab-06 backend stamps mock tags onto every post)

```
cd lab-files/lab-06/social-media/server
npm install
npm run dev
```

Terminal 2 – client

```
cd lab-files/lab-06/social-media-rr-v7
npm install
npm run dev
```

Visit <http://localhost:5173>. Sign up, log in, browse around. Let the app feel slow before you measure.

Make a backup

```
cp -R lab-files/lab-06 lab-files/lab-06-backup
```

Windows PowerShell:

```
Copy-Item -Recurse lab-files/lab-06 lab-files/lab-06-backup
```

Phase 1 — Baseline measurement (15 min)

Don’t fix anything yet. Measure first.

Step 1 — React DevTools Profiler

1. Open React DevTools → Profiler tab
2. Click record
3. Type a few characters into the search box on the home page
4. Click stop
5. Look at the flame graph. Sort by “ranked” view to see the slowest components
6. Note the top three components by render time (or render count)

Step 2 — Lighthouse

1. Open Chrome DevTools → Lighthouse tab
2. Select “Performance” only, “Mobile” device, “Simulated throttling”
3. Click “Analyze page load”
4. Note the Performance score, LCP, INP, CLS, TTFB
5. Take a screenshot or save the report

Step 3 — Bundle size

```
npm run build
ls -lh dist/assets/*.js # macOS / Linux
dir dist\assets\*.js   # Windows PowerShell
```

Note the largest JS chunk size.

Step 4 — Capture the baseline

Create lab-files/lab-06/lab06-baseline.md:

```
# Lab 6 baseline
```

Captured on `<date>` against ``lab-files/Lab-06/social-media-rr-v7``.

```
## React Profiler — top 3 components by impact
```

Component	Render count	Avg render time
PostsFeed	6	22ms
PostCard	360 (60 posts × 6 renders)	3.5ms each
TrendingSidebar	6	28ms

```
## Lighthouse (mobile, simulated)
```

Metric	Value
Performance	64
LCP	4.1s
INP	320ms
CLS	0.04
TTFB	0.8s

```
## Bundle size
```

Chunk	Size
index-XXXXX.js	412 KB

Use real numbers. Mine are made up — yours will differ.

Phase 2 — Pick three optimizations and apply them (45 min)

Look at your baseline. Pick three of the optimizations below — the ones that map to your worst numbers. Each optimization gets ~15 minutes.

Optimization options

You can pick any three. The list is roughly ordered by what tends to apply to this app.

A. Lift the search filter UP — and the post list DOWN

The home page renders a search input AND the posts list. The search state lives in the parent, so every keystroke re-renders the entire posts list. Pull the search input out into its

own component with its own state, and have it write to a URL search param that the posts list subscribes to (with a debounce on the URL write).

B. Memoize the trending sidebar's expensive calculation

TrendingSidebar recomputes a tag-frequency analysis on every render. If you've enabled the React Compiler, check whether it actually memoized — if not (eslint will tell you), add a `useMemo` manually with stable deps.

C. Lazy-load the composer

The post-creation form imports a “heavy” module that adds ~80KB. Most users land on the home page and never open the composer. Wrap the composer in `React.lazy` and a `Suspense` boundary so its bundle doesn't ship with the home page.

D. Enable the React Compiler

If the project doesn't already have it enabled, install `babel-plugin-react-compiler` and add it to the Vite config. Enable the eslint plugin too. Re-run the profiler — many of the manual `useMemos` in the codebase should now be redundant.

E. Throttle / debounce a noisy event source

If you've added an infinite-scroll trigger or any scroll-driven behavior, use `react-intersection-observer` (or a 100ms throttle) so the handler doesn't fire on every scroll event.

How to do each one

For each optimization:

1. Implement the change
2. Re-measure (Profiler if it's a render fix, Lighthouse + build if it's a bundle fix)
3. Note the before/after numbers in your notes (you'll roll them up into `lab06-results.md` next)

Phase 3 — Write up `lab06-results.md` (15 min)

Pull together what you did. Save as `lab-files/lab-06/lab06-results.md`:

Lab 6 results

What I changed

1. ****<Optimization name>**** – brief description of the change.
2. ****<Optimization name>**** – brief description.
3. ****<Optimization name>**** – brief description.

Before / after

Metric	Before	After	Change
Index bundle	412 KB	318 KB	-23%
LCP (mobile)	4.1s	3.4s	-17%
Posts component avg render	18ms	6ms	-67%
PostCard render count (per keystroke)	60	0	search no longer re-renders posts

What I tried that didn't help

(If anything – sometimes optimizations don't move the numbers, and that's worth recording.)

What I'd do next

(Usually one or two more optimizations you didn't have time for.)

The “what didn't help” section is the maturity test. Real performance work has dead ends; pretending otherwise is the AI tell.

Stretch — virtualized posts feed

Add `@tanstack/react-virtual` and convert the posts feed to a virtual list. The benefit will be real if your seeded data has 200+ posts.

```
npm install @tanstack/react-virtual
```

See `solutions/lab-06-perf/app/components/PostsFeed.jsx` for a reference implementation.

Re-profile. The render count for `PostCard` should drop dramatically — only the visible cards (plus the overscan) should render.

Hints if you get stuck

- **The Profiler shows renders but you don't see why:** click any flame-graph bar; the right panel shows “Why did this render?” listing each prop and hook that changed.
- **Lighthouse score is the same after a fix:** Lighthouse is noisy. Run it three times and average. Or use the “Throttling: applied” mode for more deterministic numbers.
- **React.lazy chunk doesn't appear in the network tab:** Vite may have inlined it because it's small. Check `dist/` after build — that's the source of truth for production behavior.
- **Compiler eslint warnings everywhere after enabling it:** that's the point. Each warning is a component the Compiler couldn't memoize. Fix them in priority order (the warning shows you why it bailed).

- **The throttled scroll handler now fires too late:** lower the throttle interval. 16ms = once per frame; 100ms = perceptibly choppy. Find the right balance.
 - **Virtualization changes layout:** the virtual container needs an explicit height, and post cards need consistent height (or you need `measureElement`). Read TanStack Virtual's docs for variable-height items.
-

Reference solution

The reference solution applies four optimizations (A, B, C, D) plus the virtualization stretch. It's at `solutions/lab-06-perf/`. Don't peek before you've finished your own — measurement habits don't transfer through reading.

Lab 7 — Test Suite from Scratch

Time: 75 minutes

Format: Hands-on. Work alone.

Goal: Set up Vitest + React Testing Library + MSW, write a baseline test suite for one feature by hand, then have an AI extend it. Mark every AI-generated test as **keep**, **refactor**, or **delete** — with a written reason. The review skill is the load-bearing part.

What you'll have at the end

1. A working Vitest + RTL + MSW test setup in the social-media client.
2. A baseline test suite (3–5 tests) for one feature, written by you, focused on behavior.
3. An AI-extended test suite (8–15 more tests).
4. A `lab07-review.md` file at the root of `lab-files/lab-07/` marking every AI-generated test as **keep**, **refactor**, or **delete** with a reason citing Module 8's red-flag list.

Plus, if you have time: one Playwright e2e test for the auth flow.

Time budget

Phase	Minutes
Setup — Vitest + RTL + MSW	15
Part A — Baseline test suite (you write)	25
Part B — AI extends the suite	15
Part C — Review every AI test	20
Stretch (if time): one Playwright e2e	+

If you finish early, do the stretch task or extend the AI-review write-up with a “what AI did well” section.

Setup

You’re working in `lab-files/lab-07/`. The starter is the post-Lab-4 state of `social-media-rr-v7/` (TanStack Query is in place) plus the shared backend.

Terminal 1 – backend

```
cd lab-files/lab-07/social-media/server
npm install
npm run dev
```

Terminal 2 – client

```
cd lab-files/lab-07/social-media-rr-v7
npm install
npm run dev
```

Verify the app still runs at `http://localhost:5173` before you start.

Make a backup

```
cp -R lab-files/lab-07 lab-files/lab-07-backup
```

Windows PowerShell:

```
Copy-Item -Recurse lab-files/lab-07 lab-files/lab-07-backup
```

Install testing deps

```
cd lab-files/lab-07/social-media-rr-v7
npm install -D vitest@2 @vitejs/plugin-react jsdom \
    @testing-library/react@16 @testing-library/user-event@14
@testing-library/jest-dom@6 \
    msw@2
```

Wire up Vitest

Create `vitest.config.js` (separate from `vite.config.js` so we don’t clobber the RR v7 starter’s plugin and dev-server config):

```
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';
```

```
export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    setupFiles: ['./app/test/setup.js'],
    globals: true,
  },
});
```

Vitest auto-detects `vitest.config.js` and uses it instead of `vite.config.js`.

Create `app/test/setup.js`:

```
import '@testing-library/jest-dom/vitest';
import { afterAll, afterEach, beforeAll } from 'vitest';
import { setupServer } from 'msw/node';
import { handlers } from './handlers';

const server = setupServer(...handlers);

beforeAll(() => server.listen({ onUnhandledRequest: 'error' }));
afterEach(() => server.resetHandlers());
afterAll(() => server.close());

export { server };
```

Create `app/test/handlers.js`:

```
import { http, HttpResponse } from 'msw';

export const handlers = [
  http.get('/api/posts', () =>
    HttpResponse.json([
      { id: 1, title: 'First post', body: 'Hello', author: 'a@b.com',
        createdAt: '2026-05-10T10:00:00Z' },
      { id: 2, title: 'Second post', body: 'World', author: 'c@d.com',
        createdAt: '2026-05-10T11:00:00Z' },
    ])
  ),
  http.post('/api/posts', async ({ request }) => {
    const body = await request.json();
    return HttpResponse.json({ id: 99, ...body, author: 'test@example.com',
      createdAt: new Date().toISOString() }, { status: 201 });
  }),
  http.post('/api/login', async ({ request }) => {
    const { email, password } = await request.json();
    if (password === 'wrong') {
      return HttpResponse.json({ message: 'Invalid email or password.' }, {
        status: 401 });
    }
    return HttpResponse.json({ token: 'fake-token', user: { id: '1', email,
      name: 'Test User' } });
  }),
  http.get('/api/me', () =>
    HttpResponse.json({ id: '1', email: 'test@example.com', name: 'Test User'
    })
  ),
];
```

Add a script to package.json:

```
"scripts": {  
  "test": "vitest"  
}
```

Run `npm test`. You should see a “no test files found” message, which is fine — you’ll add tests in Part A.

Part A — Baseline test suite, written by you (25 min)

Pick **one** feature to test. Suggested options:

- **PostsFeed** — pure presentation; renders a list of posts; shows an empty state; styles sending differently
- **NewPostForm** — validates input, submits via TanStack Query mutation, optimistic update, error path

Write **3–5 behavior-focused tests** for that feature. Save them as `<Feature>.test.jsx` next to the component.

Rules for your hand-written suite

- Query by **role first**, then label, then text. Never `getByTestId` unless you’ve exhausted the alternatives.
- Use `userEvent`, never `fireEvent`. Always `await` it.
- Each test asserts on what the user sees — visible text, focused element, navigation.
- Don’t spy on hooks or internal state.
- Don’t use `setTimeout` for waits. Use `findBy` or `waitFor`.

Example test (LoginForm — your version may differ)

```
import { describe, it, expect } from 'vitest';  
import { screen } from '@testing-library/react';  
import userEvent from '@testing-library/user-event';  
import { renderWithProviders } from '../test/renderWithProviders';  
import NewPostForm from './NewPostForm';  
  
describe('NewPostForm', () => {  
  it('disables the submit button when the body is empty', () => {  
    renderWithProviders(<NewPostForm />);  
    expect(screen.getByRole('button', { name: /post/i })).toBeDisabled();  
  });  
  
  it('clears the inputs after a successful submit', async () => {  
    const user = userEvent.setup();  
    renderWithProviders(<NewPostForm />);  
    await user.type(screen.getByPlaceholderText(/what's on your mind/i), 'hi there');
```

```
    await user.click(screen.getByRole('button', { name: /post/i }));
    expect(await screen.findByPlaceholderText(/what's on your
mind/i)).toHaveValue('');
  });
});
```

(You'll want a `renderWithProviders` helper that wraps your component in `QueryClientProvider` + `MemoryRouter`. See `solutions/lab-07-tests/app/test/renderWithProviders.jsx` for a reference if you want to peek at the shape — but write your own first.)

Run `npm test` and confirm your suite passes.

Part B — Have AI extend the suite (15 min)

Step 1 — Pick your AI assistant

Same rule as Lab 1 — use whatever you have.

Step 2 — Give it the test file and a clear instruction

Try a prompt like:

Here's my baseline test suite for [feature]. Add 8 more tests covering edge cases and additional behaviors. Use Vitest + React Testing Library + MSW. Query by role/label/text, not test IDs. Use user-event, not fireEvent. Keep each test focused on a single behavior. Follow the conventions in AGENTS.md.

The instructions matter — without the role/label/user-event guidance, AI will default to `getByTestId` and `fireEvent`.

Step 3 — Don't read the AI output yet

Just paste it in. We're going to review it in Part C.

Run `npm test`. Some of the AI tests will fail, some will pass. That's expected. **Don't fix the failures yet** — failures are part of what you're reviewing.

Part C — Review every AI test (20 min)

This is the load-bearing part of the lab. Open `lab-files/lab-07/lab07-review.md` and create a section for every AI-generated test.

Per-test format

For each test the AI added, decide: **keep, refactor, or delete** — and write a one-sentence reason. Cite the Module 8 red-flag list when applicable.

AI test #1 – "renders the post list"

****Verdict:**** keep

****Reason:**** queries by role, asserts on visible text, single behavior. Solid baseline test.

AI test #2 – "calls setLoading when fetching"

****Verdict:**** delete

****Reason:**** spies on internal state via `vi.spyOn`` (Module 8 red flag #1).

Tests

implementation. The behavior – that the loading state shows – is already covered

by AI test #1's render assertion.

AI test #3 – "shows the search results after typing"

****Verdict:**** refactor

****Reason:**** uses `fireEvent.change`` instead of `userEvent.type`` (red flag #4).

Refactored to use userEvent and findBy. Updated test passes.

Red flags from Module 8

Use these when justifying a “delete” or “refactor” verdict:

1. `vi.spyOn` on React hooks — testing internals
2. `getByTestId` everywhere instead of `getByRole`
3. `expect(setState).toHaveBeenCalledWith(...)` — implementation coupling
4. `fireEvent` instead of `userEvent`
5. Mocking individual functions in `api.js` — should mock at the network
6. `setTimeout`-based waits — flakiness factory
7. Tests that pass without rendering anything — checking pure utils only
8. Snapshot tests for entire components

Apply your verdicts

For each test:

- **Keep** — leave it alone
- **Refactor** — rewrite it according to your reason. Test should still pass.
- **Delete** — remove it from the file

Run `npm test` again. All remaining tests should pass.

Stretch — one Playwright e2e

If you have time, add a single Playwright test for the login flow.

```
npm install -D @playwright/test
npx playwright install chromium
```

Create e2e/login.spec.js:

```
import { test, expect } from '@playwright/test';

test('user can sign in and reach the home feed', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel(/email/i).fill('test@example.com');
  await page.getByLabel(/password/i).fill('correctpass');
  await page.getByRole('button', { name: /sign in/i }).click();

  await expect(page).toHaveURL(/\/home/);
  await expect(page.getByRole('heading', { name: /recent posts/i
})).toBeVisible();
});
```

See solutions/lab-07-stretch/playwright.config.js for a reference config. The webServer block boots Vite for you so you don't need a separate startup step.

Hints if you get stuck

- **Error: Element type is invalid when rendering:** check that you wrap the component in any required providers (QueryClientProvider, MemoryRouter, etc.). Create a renderWithProviders helper if every test needs the same wrapping.
- **Unable to find an accessible element with role 'button':** the component might render buttons as <div onClick>. That's a real accessibility bug — fix the component, not the test.
- **MSW handler not matching:** check the URL exactly. Trailing slashes matter. Use onUnhandledRequest: 'error' in test setup so unmocked requests fail loudly.
- **act warnings in console:** you forgot to await a userEvent or a state update. The warning shows you the line.
- **AI test uses vi.useFakeTimers():** usually unnecessary in a behavior test. Refactor to use findBy or remove.
- **AI test imports your component's hooks directly to spy on them:** definitely refactor or delete.

Reference solutions

- solutions/lab-07-tests/ — main solution with lab07-review.md showing the kind of judgment to aim for

- [solutions/lab-07-stretch/](#) — Playwright stretch

Lab 8 — AI-Assisted Feature, End to End

Time: 90 minutes

Format: Hands-on. Work alone.

Goal: Pick a feature from the backlog. Write a one-paragraph spec. Build it with heavy AI assistance. Self-review using the checklist from Module 9.

This is the capstone. Everything from this week comes together: hooks, routing, state, data, RSC, performance, testing, AI review.

What you'll have at the end

1. A `lab08-spec.md` — your one-paragraph spec for the feature.
2. Working feature code in `lab-files/lab-08/`.
3. A `lab08-self-review.md` listing at least three issues you caught and fixed in the AI's output, citing Module 9's review checklist.

Both `.md` files go at the root of `lab-files/lab-08/`.

Optional: one entry contributed to the class's `ai-mistakes-we-saw.md` doc.

Time budget

Phase	Minutes
Pick a feature + write the spec	15
Set up project context for AI	10
Build with AI — small batches	45
Self-review against the checklist	20

If you finish early, contribute to the class's `ai-mistakes-we-saw.md` doc with a pattern you noticed.

Setup

You're working in `lab-files/lab-08/`. The starter is the post-Lab-7 state of `social-media-rr-v7/` (TanStack Query + tests in place) plus the shared backend.

Terminal 1 — backend

```
cd lab-files/lab-08/social-media/server
```

```
npm install
npm run dev
```

Terminal 2 - client

```
cd lab-files/lab-08/social-media-rr-v7
npm install
npm run dev
```

If you'd rather extend the Next.js client, switch to `lab-files/lab-02/social-media-nextjs/` (it's the only Next client we ship). The lab is framework-neutral.

Make a backup

```
cp -R lab-files/lab-08 lab-files/lab-08-backup
```

Windows PowerShell:

```
Copy-Item -Recurse lab-files/lab-08 lab-files/lab-08-backup
```

Phase 1 — Pick a feature + write the spec (15 min)

Backlog

Pick one. They're roughly ordered by complexity — the lower ones are bigger.

A. Comments on posts. Each post has a thread of replies. Show count on the post card; expand on click. Server Action to add a comment. Optimistic update.

B. Reactions on posts. Heart, laugh, surprise. Each reaction is a Server Action. The post card shows aggregate counts and which reactions the current user has applied. Toggle via click.

C. Tags + tag filtering. Posts can have up to 5 tags. The home feed filters by clicked tag. Tags are URL-driven. Show a tag cloud in the sidebar.

D. User profile edit. A `/profile/edit` route with a form for name, bio, avatar URL. Uses a Server Action with validation. Shows the form pre-filled from the current user.

E. Saved drafts. Compose a post but don't send it; it goes to drafts. List of drafts in the sidebar; click to resume composing. Drafts persist in `localStorage` AND sync to the server when online.

F. Notifications panel. A bell icon in the header. Clicking opens a panel with recent notifications (someone replied, someone reacted). Mark-as-read state syncs to the server.

G. Direct messages. A `/messages` route. List of conversations on the left; thread on the right. Send messages via Server Action. Optimistic updates.

H. Search with suggestions. Search box with as-you-type suggestions. Debounced. Results page at `/search?q=...` Highlight match in results.

If none of these fit, propose your own — but get the instructor’s nod before you start.

Write the spec

Save as lab-files/lab-08/lab08-spec.md. One paragraph. Answer all of these:

1. **What does the feature do?** One sentence.
2. **What are the acceptance criteria?** Three to five bulleted user-facing behaviors.
3. **Where in the codebase does it live?** File paths or directory paths.
4. **What’s explicitly out of scope?** Anything you’d be tempted to scope-creep into.

Example for option B (Reactions):

Lab 8 spec – Reactions on posts

What it does

Each post supports three reactions (heart, laugh, surprise). Users can toggle each independently.

The post card shows aggregate counts per reaction and highlights ones the current user applied.

Acceptance criteria

- Clicking a reaction icon adds it for the current user (optimistic).
- Clicking again removes it.
- Counts update in the UI immediately and persist to the server.
- Other users' reactions appear without a manual refresh (TanStack Query refetch on focus is fine).
- Server rejects invalid reaction types with a 400 status.

Where it lives

- Frontend: app/components/PostCard.jsx (extend), app/components/ReactionBar.jsx (new)
- Server Actions: app/actions/reactions.js (new)
- API: server/routes/reactions.js (new)

Out of scope

- Custom reaction emoji
- Reaction history / who-reacted modal
- Notifications when someone reacts to your post (that's option F)
- Animations beyond simple state changes

A spec that doesn’t fit on one screen is too big. Trim or split.

Phase 2 — Set up project context for AI (10 min)

The AI will produce dramatically better code if it knows your codebase’s conventions. Spend ten minutes priming it.

Step 1 — Verify AGENTS.md / CLAUDE.md exists

Read AGENTS.md (or CLAUDE.md) at the repo root. Confirm it accurately describes the current stack: React 19, RR v7 framework mode, TanStack Query, Zustand, Vitest + RTL + MSW.

If it doesn't exist or is stale, update it now. The reference "Example AGENTS.md" slide from Module 9 is a good template — and yes, you can have the AI write it for you (it just read the same codebase).

Step 2 — Open the lint config

Confirm react-hooks/exhaustive-deps, react-compiler/react-compiler, and the Testing Library plugin are all set to error. AI tools listen to lint errors and self-correct when they see them.

Step 3 — Drop the spec into the AI's context

Either paste it into the chat panel, or tell the AI to read lab-files/lab-08/lab08-spec.md from disk. Add: "Don't start coding yet. First, propose a file list and function signatures. Wait for me to approve."

The AI should produce a structure proposal. **Read it carefully.** This is your chance to redirect before any code is written.

If the AI proposes class components, getByTestId, useEffect for fetching, or 'use client' on a page that doesn't need it — push back. "We use [tool], not that. See AGENTS.md."

Step 4 — Approve or revise the structure

Once the AI's proposed structure matches your codebase's conventions, tell it to proceed. Keep the iterations small from here.

Phase 3 — Build with AI — small batches (45 min)

The rhythm

For each piece of the feature:

1. Tell the AI which file to write and what it should do
2. Read the diff before accepting
3. Run `npm run dev` to confirm it works
4. Run `npm test` (if there are tests for the area)
5. Move to the next file

Order

Approximately:

1. The data layer first (Server Action, API route, mocks)
2. The lowest-level component (e.g., the icon bar before the card that uses it)
3. The integration component (the post card, the page)
4. Tests for the new component
5. Manual smoke test in the browser

What to push back on

If the AI:

- Adds `'use client'` to a page that doesn't need it — push back, extract the interactive bit
- Uses `useEffect` + `fetch` instead of a Server Action or TanStack Query — push back
- Adds `getByTestId` or `fireEvent` in tests — push back
- Imports a server-only module into a `'use client'` file — push back, refactor
- Suggests `localStorage` for anything sensitive — push back, use server state

What to accept

If the AI:

- Names files consistently with your repo
- Uses your existing helpers (`api.js`, `queryKeys.js`, etc.)
- Writes tests using `getByRole` and `userEvent`
- Asks clarifying questions before assuming

You're building the right working relationship — keep going.

Phase 4 — Self-review against the checklist (20 min)

Open Module 9's "AI-generated React review checklist" slide and walk every file you and the AI produced through it.

The checklist (copy from Module 9)

Hooks - Stale closures in `useEffect`? - Missing dep array entries? - Hooks inside conditionals or loops? - `useEffect` for derived state or events?

Boundaries - `'use client'` at the right level? - Server-only imports staying server-side? - Functions across boundary marked `'use server'`? - Non-serializable props eliminated?

Patterns - Modern Router APIs (RR v7 / Next.js App Router)? - Modern state (Zustand/Jotai/RTK or just useState)? - Server data through TanStack Query, not useEffect+fetch? - Functional setState where it matters?

Tests - Role-based queries, not getByTestId? - user-event, not fireEvent? - MSW for network mocks, not vi.fn on api? - No spies on React internals?

Write lab08-self-review.md

Save at lab-files/lab-08/lab08-self-review.md. Format:

Lab 8 self-review

Reviewed against Module 9's checklist. Found and fixed N issues.

Issue 1 – <short description>

```
**Where:** path/to/file.jsx, lines NN-NN
**Category:** hooks / boundaries / patterns / tests
**What the AI produced:**
``js
// the bad code, copy-pasted
```

What I changed it to:

```
// the corrected code
```

Why: one or two sentences.

Issue 2 — ...

Issue 3 — ...

You need at least three real issues. If your AI produced perfect code on the first try (rare), explain what about your spec or context made that possible – that's a finding too.

Optional – contribute to `ai-mistakes-we-saw.md`

If you noticed a pattern this week that's not in Module 9's taxonomy, add an entry to the class's shared `ai-mistakes-we-saw.md` doc.
Format:

```
```markdown
<pattern name>
```

```
What you'll see: <description of the mistake>
Why AI does it: <your guess at the cause – training data? prompt
```

```
phrasing?>
How to spot it: <what makes it visible in code review>
The fix: <what to do instead>
```

These contributions feed into the next cohort's Module 9.

---

### Hints if you get stuck

- **The AI's first proposal is wildly off-spec:** your spec was probably ambiguous. Tighten it. Sometimes "what's out of scope" is the missing piece — without that, the AI will fill in the gaps with its own assumptions.
- **The AI keeps producing the same kind of mistake:** it's pattern-matching to its training data, not to your codebase. Add an explicit "do not" rule to AGENTS.md and prompt the AI to re-read it.
- **You don't have time to finish:** ship the spec, half the implementation, and the self-review of what you did get to. The review skill is what's being graded — finishing every line of code is secondary.
- **The AI added tests you can't make pass:** that's a self-review item. Either fix the tests OR fix the code OR delete the tests with a written reason. All three are valid; "leave failing tests in CI" is not.
- **You're not sure if the AI's choice is wrong:** that's also a self-review item. "Considered alternatives X and Y; AI's choice is reasonable because Z." Documenting reasoning is the skill.

---

### Reference solution

The lab is open-ended by design — there isn't one specific solution. The `solutions/lab-08-exemplar/` folder implements option B (Reactions) with a benchmark `lab08-self-review.md` showing the kind of triage to aim for.